

Day 2 – SystemVerilog Integer, Vector Types & Literals

1. Integer Data Types

SystemVerilog integer types differ in width, 2-state/4-state behavior, and signedness. Common types:

Type	Width	States	Default
bit	1	2	unsigned
logic	user-defined	4	unsigned
reg	user-defined	4	unsigned
byte	8	2	signed
shortint	16	2	signed
int	32	2	signed
longint	64	2	signed
integer	32	4	signed

2. Packed Arrays / Vectors

Packed dimensions are written **before** the variable name. They group bits into one vector.

```
logic [7:0] data;  
data = 8'b1010_1100;  
data[7] = 1, data[0] = 0, data[7:4] = 1010.
```

Multiple packed dimensions are possible: `logic [1:0][7:0] data;` → $2 \times 8 = 16$ packed bits.

3. Unpacked Arrays

Unpacked dimensions are written **after** the variable name and create separate elements.

```
logic data [0:7];  
logic [7:0] memory [0:255];
```

The second declaration has 256 elements, each containing 8 packed bits. `memory[10]` selects one 8-bit location; `memory[10][7]` selects bit 7 of that location.

Memory rule: Before the variable name = PACKED; after the variable name = UNPACKED.

4. Number Literals

A literal is a value written directly in code. Common form: `<size>'[s]<base><value>`. The size is optional; **s** explicitly means signed.

Example	Meaning
8'b1010_1010	8-bit binary
8'o252	8-bit octal
8'd170	8-bit decimal
8'hAA	8-bit hexadecimal
8'sd-5	8-bit signed decimal
8'shFF	8-bit signed hexadecimal

5. Bases & Readability

Binary uses 0/1, octal uses 0–7, decimal uses 0–9, and hexadecimal uses 0–9/A–F. Decimal 170 can be written as `8'b1010_1010`, `8'o252`, `8'd170`, or `8'hAA`. Underscores improve readability and do not change the value.

6. Sized vs Unsized Literals

`8'd25` is explicitly 8-bit (sized). `25` is an unsized integer literal. When width matters, sized literals make the intended hardware width clearer.

7. Signed vs Unsigned Literals

8'hFF is an unsigned 8-bit based literal; 8'shFF is explicitly signed. The bit pattern 1111_1111 represents 255 unsigned but -1 as signed 8-bit two's complement.

If a variable is declared `logic signed [7:0] a;`, simple assignment does not require a signed literal: `a = 8'd25;` is valid. The **s** controls the literal; **signed** in the declaration controls the variable.

8. X and Z Literals

4-state types can represent unknown and high-impedance values: 8'bxxxx_xxxx, 8'bzzzz_zzzz, or mixed values such as 8'b10xx_zz01.

9. Unbased Unsized Literals

'0, '1, 'x, and 'z are useful for filling a destination/context.

```
logic [7:0] data;
data = '0; // 0000_0000
data = '1; // 1111_1111
data = 'x; // xxxx_xxxx
data = 'z; // zzzz_zzzz
```

10. Arithmetic & Overflow

An N-bit unsigned value ranges from 0 to 2^N-1 . An N-bit signed two's-complement value ranges from $-2^{(N-1)}$ to $2^{(N-1)}-1$. If a result does not fit the destination width, higher bits can be lost.

```
logic [7:0] a; a = 8'd255; a = a + 1; → stored 8-bit result wraps to 0.
logic signed [7:0] b; b = 8'sd127; b = b + 1; → 8-bit signed overflow produces -128.
```

11. Signed / Unsigned Comparisons

Signedness changes numerical interpretation. With 8 bits, 1111_1111 is 255 unsigned but -1 signed; 1000_0000 is 128 unsigned but -128 signed. Always check signedness before comparisons.

Day 2 Quick Checklist

- Identify type, width, state model, and signedness.
- Remember: brackets before name = packed; after name = unpacked.
- Read literal syntax: size + optional s + base + value.
- Know binary, octal, decimal, and hexadecimal.
- Understand sized/unsized literals and underscores.
- Know X/Z and '0/'1/'x/'z.
- Predict overflow and signed/unsigned comparison behavior.